LITHO MAINNET 9005 · EXCHANGE INTEGRATION

LEP100 Mainnet Exchange Integration

Last reviewed: 2026-07-29

Status: **standard documented; mainnet token registry pending**. No LEP100 contract address is approved for exchange listing on EVM chain `9005` in this repository at the time of review. Testnet addresses must never be reused as mainnet asset identifiers.

1. What LEP100 means on mainnet

The current reference implementation is an EVM fungible token built from OpenZeppelin Contracts 5.x:

- `ERC20`
- `ERC20Burnable`
- `Ownable`

Reference source: `contracts/contracts/LEP100Token.sol`.

Property	Reference behavior
Solidity version	`0.8.24`
Source SHA-256	`d3153fd7473498c872913eb0cf213dae61cd608048d815cf7211b3cb835dfdc1`
Supply	Fixed at construction; minted to deployer
Decimals	Immutable constructor value; query per token
Transfer	Standard ERC20 `transfer` / `transferFrom`
Burn	Holder `burn`; allowance-based `burnFrom`
Mint after construction	Not present
Pause/blacklist/tax/rebase	Not present
Proxy/upgrades	Not present in reference contract
Permit	Not present; no `ERC20Permit` inheritance
Gas coin	Native LITHO

`Ownable` does not grant a hidden mint or freeze function because the reference contract defines no owner-only operational methods. Ownership can still be transferred or renounced through inherited `Ownable` functions.

Any modified deployment must be reviewed as a separate token contract. A name or symbol containing "LEP100" does not prove conformance.

2. Constructor and amount units

```
constructor(
    string memory name_,
    string memory symbol_,
    uint8 decimals_,
    uint256 totalSupply_
)
```

The constructor treats `totalSupply_` as whole display tokens and internally mints:

`totalSupply_ * 10 ** decimals_` base units.

An exchange must query and store the resulting on-chain `totalSupply()` and `decimals()` values; do not reconstruct them from issuer marketing material.

3. Mainnet listing identifier

The unique asset key must include:

```
network = lithosphere-mainnet
evm_chain_id = 9005
contract_address = 0x...
```

Never identify a token by symbol alone. Symbols and names can collide. Store the checksummed contract address and its bytecode hash.

4. Required listing dossier

Before enabling a LEP100 asset, obtain:

- exact mainnet contract address on chain `9005`;
- deployment transaction hash and deployment block;
- issuer/legal entity and authorized contact;
- name, symbol, decimals, and total supply;
- verified Solidity source, compiler version, optimizer settings, and constructor arguments;
- source and deployed-bytecode hashes;
- ownership address and ownership/renouncement policy;
- distribution and treasury addresses;
- token-specific audit report, if required by the exchange;
- logo/metadata separately from the on-chain identity;
- deposit minimum, withdrawal fee, and confirmation policy; and
- explicit statement whether the contract is the unmodified reference or a modified implementation.

The production explorer is not yet available for source verification. Until it is approved, verification evidence must come from reproducible compilation and direct RPC code comparison.

5. On-chain conformance checks

Run all calls against the exact proposed contract:

Method	Selector	Requirement
`name()`	`0x06fdde03`	ABI string; informational only
`symbol()`	`0x95d89b41`	ABI string; not a unique ID
`decimals()`	`0x313ce567`	uint8; configure accounting exactly
`totalSupply()`	`0x18160ddd`	uint256 base units
`balanceOf(address)`	`0x70a08231`	uint256 base units
`transfer(address,uint256)`	`0xa9059cbb`	standard successful transfer behavior
`allowance(address,address)`	`0xdd62ed3e`	uint256
`approve(address,uint256)`	`0x095ea7b3`	standard approval behavior
`transferFrom(address,address,uint256)`	`0x23b872dd`	allowance-based transfer

Also verify:

1. `eth_getCode` is non-empty at the listing block and current head.
2. The deployment receipt succeeded.
3. Reproducibly compiled runtime bytecode matches the deployed runtime code, accounting for linked/immutable metadata as appropriate.
4. A test transfer produces exactly the expected recipient balance delta and a standard `Transfer` event.
5. No proxy storage slot or delegatecall upgrade mechanism exists unless the exchange has explicitly approved an upgradeable token.
6. No fee-on-transfer, reflection, rebasing, blacklist, pause, or hidden mint behavior exists.

6. Minimal exchange ABI

```
[
  {"type":"function","name":"name","stateMutability":"view","inputs":[],"outputs":[{"type":"string"}]},
  {"type":"function","name":"symbol","stateMutability":"view","inputs":[],"outputs":[{"type":"string"}]},
  {"type":"function","name":"decimals","stateMutability":"view","inputs":[],"outputs":[{"type":"uint8"}]},
  {"type":"function","name":"totalSupply","stateMutability":"view","inputs":[],"outputs":[{"type":"uint256"}]},
  {"type":"function","name":"balanceOf","stateMutability":"view","inputs":[{"name":"account","type":"address"}],"outputs":
[{"type":"uint256"}]},
  {"type":"function","name":"transfer","stateMutability":"nonpayable","inputs":[{"name":"to","type":"address"},
{"name":"value","type":"uint256"}],"outputs":[{"type":"bool"}]},
  {"type":"event","name":"Transfer","anonymous":false,"inputs":[{"name":"from","type":"address","indexed":true},
{"name":"to","type":"address","indexed":true},{"name":"value","type":"uint256","indexed":false}]}
]
```

Use the full verified ABI for security review; this minimal ABI is only for routine exchange balance, transfer, and deposit operations.

7. Deposit detection

The standard event is:

```
Transfer(address indexed from, address indexed to, uint256 value)
topic0 = 0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef
```

For every candidate deposit:

1. Require EVM chain ID `9005`.
2. Require log `address` to equal the exact allowlisted token contract.
3. Require `topic0` to equal the `Transfer` signature.
4. Decode indexed `to` and require the assigned exchange deposit address.
5. Decode `value` as a uint256 integer.
6. Fetch and require receipt `status == 0x1`.
7. Verify receipt block hash against the canonical block.
8. Wait the exchange-approved confirmation buffer.
9. Credit by `(9005, transaction_hash, log_index)`.

Do not credit from input calldata. A reverted transaction can contain valid- looking `transfer` calldata but moves no tokens. A single transaction can emit multiple `Transfer` logs, so transaction hash alone is not a sufficient idempotency key.

8. Withdrawals and token sweeping

Use standard `transfer(recipient, amount)` from the custody address. Before signing:

- validate chain ID `9005`;
- validate recipient address and reject zero address;
- ensure `amount` is an integer base-unit value;
- query current token balance;
- estimate gas against the exact contract call;
- simulate with `eth_call` where custody tooling permits;
- reserve native LITHO for gas; and
- enforce nonce, fee, and withdrawal policy in the offline/HSM signer.

Each token-holding deposit address needs native LITHO to sweep tokens. Plan a controlled gas-funding workflow and prevent gas-dust abuse. Token fees cannot normally be paid in the LEP100 token itself.

9. Decimals and accounting

If `decimals() = d`:

`display amount = base-unit integer / 10^d`.

Use arbitrary-precision integers and decimal formatting. Never use IEEE-754 floating point for balances, credits, or withdrawals. Reject a withdrawal that cannot be represented exactly in the token's configured base units.

Monitor `decimals`, `totalSupply`, bytecode, and ownership state after listing. The reference decimals are immutable, but monitoring detects contract/address substitution and unreviewed implementations.

10. Common transfer types

- Direct holder transfer: `transfer(to, amount)`
- Allowance-based transfer: `approve(spender, amount)` then `transferFrom(from, to, amount)`
- Burn: `burn(amount)`
- Allowance-based burn: `burnFrom(account, amount)`

The reference contract does not contain batch transfer, batch-all, mint, pause, freeze, tax, blacklist, rebase, permit, or proxy upgrade functions.

11. Finality and replay

LEP100 transactions inherit Lithosphere's CometBFT finality. The provisional exchange recommendation is 20 blocks for routine deposits and 100 for high-value deposits, subject to formal client/exchange approval.

The indexer must be able to replay a block range without double crediting. Persist canonical block hashes and revalidate them during recovery. Pause crediting if the node stalls, chain IDs differ, block hashes conflict, or the token code changes.

12. Cross-chain and wrapped-token warning

MultX and the mainnet bridge are disabled. No destination-chain wrapped LEP100 or wrapped LITHO address is approved in this package, and there is no completed independent bridge audit report to provide.

Do not accept:

- Makalu/Kamet LEP100 contract addresses;
- old `wLITHO` testnet contracts;
- an issuer-provided token on Ethereum, BNB, Base, or another network as equivalent to the mainnet asset; or
- bridge mint/burn events as deposits without a separately approved bridge integration and audit.

13. Mainnet registry required before listing

The project must publish a machine-readable mainnet registry containing, for each approved token:

```
{
  "network": "lithosphere-mainnet",
  "chainId": 9005,
  "contract": "0x...",
  "name": "...",
  "symbol": "...",
  "decimals": 18,
  "deploymentTx": "0x...",
  "runtimeCodeHash": "0x...",
  "sourceRelease": "...",
  "status": "active"
}
```

It must be signed or delivered from an authenticated official release channel. Explorer discovery alone must not add an asset to an exchange allowlist.

14. References

- Local reference source: `contracts/contracts/LEP100Token.sol`

- OpenZeppelin Contracts 5.x ERC20: `https://docs.openzeppelin.com/contracts/5.x/api/token/erc20`
- Ethereum JSON-RPC: `https://ethereum.org/developers/docs/apis/json-rpc/`
- Project API examples: MAINNET_EXCHANGE_API_REFERENCE.pdf

Generated from controlled Markdown sources · 2026-07-29